॥ श्री गणेशाय नमः ॥

Machine Learning

Simple Linear Regression:

$\hat{y} = mx + c$ $\quad m = \frac{N\Sigma x_i y_i - (\Sigma x_i)(\Sigma y_i)}{N\Sigma x_i^2 - (\Sigma x_i)^2}$ $\quad c = \frac{\Sigma y_i - m\Sigma x_i}{N}$ ← closed form solution using least squares

or solve $\bar{y} = a + b\bar{x}$; $\overline{xy} = a\bar{x} + b\overline{x^2}$] simultaneously.

Using gradient descent: $\theta = \theta - \eta \frac{\partial L}{\partial \theta}$ ← update rule

} Level curve in LR only has 1 local/global minima.

At each update, we move opp to the gradient.

for a 3-D curve:

$\nabla f \rightarrow$ grad, $u \rightarrow$ direction of interest

$\frac{\partial F}{\partial u} = \nabla f \cdot \hat{u} = |\nabla f||\hat{u}| \cos\theta$ → +1 (steepest ascent $\theta = 0°$), -1 (steepest descent $\theta = 180°$)

Direction of gradient is ⊥ to the contour curve.

- The SLR line always passes through $\bar{x}, \bar{y}$.
- Parameters in SLR are unbiased.
- confidence interval: $m \in \left[\hat{m} \pm t_{\frac{\alpha}{2}, n-2}\, SE(\hat{m})\right]$ $\quad SE(\hat{m}) = \frac{\sigma_{\hat{m}}}{\sqrt{\Sigma(x_i - \bar{x})^2}}$
- Assumptions — linearity; Independence of residual errors (0 correlation); Homoscedasticity (constant variance of errors); Normality of residuals

Multiple linear Regression: $\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p + \varepsilon$

$\hat{y} = \beta X + \varepsilon$

closed form solution to minimize RSS → $\beta = (X^T X)^{-1} X^T Y$

multicollinearity: 2/more feature variables are highly correlated.

Variance inflation factor $VIF = \frac{1}{1 - R_i^2}$ $\quad R_i^2 \rightarrow R^2$ calculated for fit $x_1 = \beta_0 + \beta_2 x_2 + \ldots$

$VIF = 1 \rightarrow$ 0 correlation

$\in (5, 10) \rightarrow$ moderate

Polynomial Regression:

Simple polynomial regression: $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_n x^n$ ← n deg. polynomial

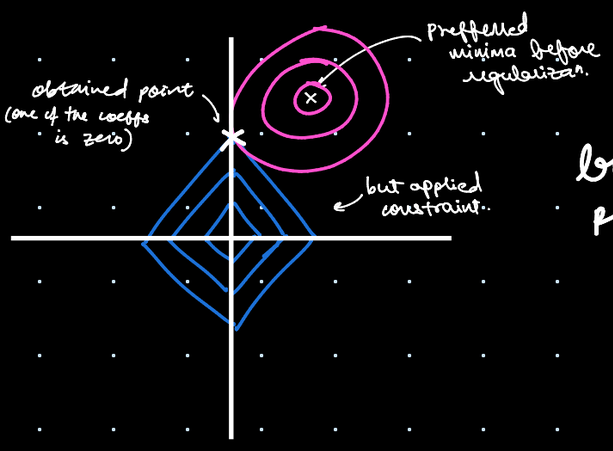
Increasing the degree may lead to overfitting.

Regularization: Add a penalty term to the loss function to reduce the model complexity by shrinking the coefficients.

LASSO (L1): $cost = SSE + \lambda \sum_{i=1}^{p} |\beta_i|$ → creates sparsity (shrinks some coeffs to 0); → feature selection; → can't apply least squares (not differentiable) use coordinate ascent.

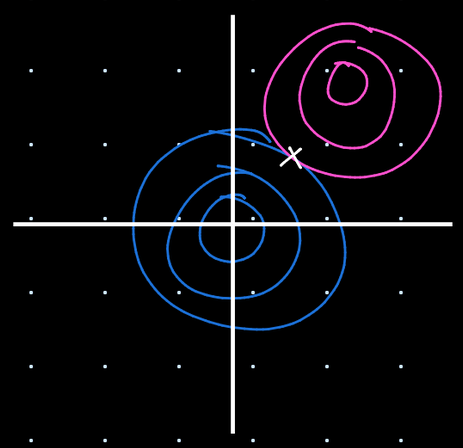
Ridge (L2): $cost = SSE + \lambda \sum_{i=1}^{p} \beta_i^2$ → not for feature selection. Distributes penalty equally among coeffs.

Elastic net: $cost = SSE + \alpha\left[l_1\text{-ratio} \cdot \sum_{i=1}^{p} |\beta_i| + (1 - l_1\text{-ratio}) \cdot \sum_{i=1}^{p} \beta_i^2\right]$



LASSO

both min at points tangential to both curves.



Ridge

Increasing regularization parameter decreases model complexity.

Regularization makes LR predictions scale variant.

closed form for ridge regression: $\beta = (X^T X + \lambda I)^{-1} X^T y$

Time complexity for MLR using the closed form solution: $O(nd^2 + d^3)$

Error metrics for regression:

- $MAE = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y_i}|$
- $SSE = \sum_{i=1}^{N} (y_i - \hat{y_i})^2$
- $MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y_i})^2$
- $RMSE = \sqrt{MSE}$
- $MAPE = \left(\frac{1}{N} \sum_{i=1}^{N} \left|\frac{y_i - \hat{y_i}}{y_i}\right| \times 100\right)\%$
  (mean absolute percentage error)
- $R^2 = 1 - \frac{\sum (y_i - \hat{y_i})^2}{\sum (y_i - \bar{y})^2}$ ← variance of y captured by x / ← variance of y

worst ↓ ↓ best

$R^2 \in [0, 1]$

- $R^2_{adjusted} = 1 - \frac{\sum (y_i - \hat{y_i})^2 / (N-K-1)}{\sum (y_i - \bar{y})^2 / (N-1)}$

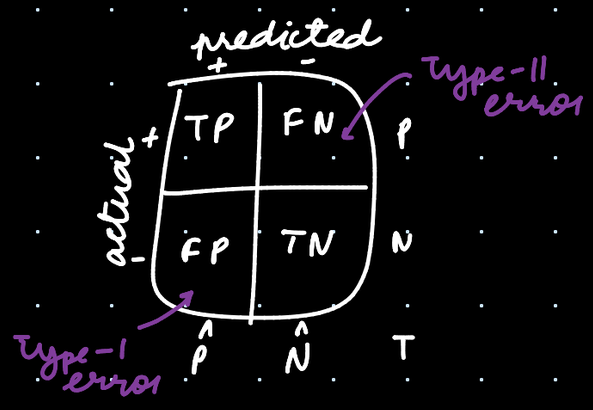
$N \rightarrow$ # of data points
$K \rightarrow$ # of predictors.

Classification:

- converting categorical data to numeric data
  - label encoding (to each category)
  - one-hot encoding (binary feature for each category)
- Handling numerical features
  - feature scaling
    - min-max
    - z-score
  - Handling outliers
    - clipping
    - log transformation
- missing data
  - removal
  - Imputation

- Softmax: $\sigma(z_i) = \frac{e^{z_i}}{\sum_{i=1}^{K} e^{z_j}}$
- Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$

Confusion matrix



- Accuracy $= \frac{TP + TN}{T}$
- Precision $= \frac{TP}{\hat{P}}$
- Recall /sensitivity (TPR) $= \frac{TP}{P}$
- Specificity (TNR) $= \frac{TN}{N}$
- f_1-score $= 2 \cdot \frac{\text{Precision} \times \text{recall}}{\text{precision} + \text{recall}}$

Averaging these for multi-class classification:

- macro-averaging: $\text{macro_precision} = \frac{\sum_{i=1}^{K} \text{Precision}_i}{K}$
- micro-averaging: $\text{micro_precision} = \frac{\text{total TP}}{\text{total TP} + \text{total FP}}$
- weighted_averaging: $\text{weighted_precision} = \frac{\sum w_i \cdot \text{Precision}_i}{\sum w_i}$ $\quad w_i$ = actual # of instances in i.

ROC curve:

(Receiver operating characteristic)

$TPR = \frac{TP}{P}$ $\quad FPR = \frac{FP}{N}$ $\quad$ ROC curve plots TPR and FPR.

AUC: Area under the curve: AUC ↑ model better at distinguishing classes.

Log loss (cross entropy loss):

$$L = \frac{-1}{N} \sum_{i=1}^{N} (y_i \ln p_i + (1-y_i) \ln(1-p_i))$$

$$L = \frac{-1}{N} \sum_{i=1}^{N} \sum_{j=1}^{K} y_{ij} \ln(p_{ij})$$

For multi-class classification:

Hamming loss: measures fraction of labels that are incorrectly predicted.

$$\text{Hamming loss} = \frac{1}{N \times K} \sum_{i=1}^{N} \sum_{j=1}^{K} 1 \cdot (y_{ij} \neq \hat{y_{ij}})$$

K-Nearest Neighbors

$x_1 \rightarrow (x_{11}, x_{12}, \ldots, x_{1N})$
$x_2 \rightarrow (x_{21}, x_{22}, \ldots, x_{2N})$

Distance metrics $d(x_1, x_2)$

→ Euclidean (L2 norm): $\sqrt{\sum (x_{1i} - x_{2i})^2}$

→ Manhattan (L1 norm): $\sum |x_{1i} - x_{2i}|$

→ Minkowski (p norm): $\left(\sum |x_{1i} - x_{2i}|^t\right)^{1/t}$

→ Cosine similarity: $\frac{\sum x_{1i} \cdot x_{2i}}{\sqrt{\sum x_{1i}^2} \sqrt{\sum x_{2i}^2}}$ $\cos\theta = \frac{\vec{a} \cdot \vec{b}}{\|a\| \cdot \|b\|}$

→ Chebyshev dist: $\max_i |x_{1i} - x_{2i}|$

→ Jaccard index (similarity) = $\frac{|A \cap B|}{|A \cup B|}$

→ Hamming distance = $\sum \mathbb{1}(x_{1i} \neq x_{2i})$

Working of KNN:
- No explicit training.
- Calculate distances of query point from all the data points.
- Find k-nearest neighbors and use majority voting.

$k \downarrow$ noise $\uparrow$ overfitting $\uparrow$ $\quad$ $k \uparrow$ noise $\downarrow$ underfit $\uparrow$

For regression: calculate inverse weighted average with k-nearest multivariate neighbors.

Distance to k^{th} nearest neighbor is a local density estimate (used as outlier score).

NOTE:
- Train set accuracy is always 100%. (Error can be non-zero only when diff. label points have the same x).
-

Naive Bayes

- Naive Bayes assump^: conditional independence of features given the label.

$$P(x_1, x_2, \dots, x_n | C_K) = \prod_i P(x_i | C_K)$$

- $P(Y = y | X_1 = x_{1i}, \dots, X_n = x_{nj}) = P(X_1 = x_{1i} | Y = y) \cdot P(X_n = x_{nj} | Y = y) \cdot P(Y = y)$

- Laplace smoothing: (used when some feature x_i takes a particular value but there's no training example like that).

$$P(\text{feature} | \text{class}) = \frac{\#\text{ of times the feature appears given the class} + \alpha}{\#\text{ of class instances} + \alpha n}$$

$\alpha \rightarrow$ often 1
$n \rightarrow$ # of diff. values the feature can take.

- Handling numerical data:
  - → Binning the values: treat them like categorical variables.
  - → Assume $P(X_i | y)$ has some distribution.
    use MLE to calculate distribution parameters.

Point prob. $P(x = x | y)$ would be 0 but we'll use whatever value the PDF gives, we don't care about that as the prob of that value. we just want comparisons.

Logistic Regression

- $\sigma(z) = \frac{1}{1+e^{-z}}$ → bounded (0,1)
  → symmetric $\sigma(z) + \sigma(-z) = 1$
  → derivative $\sigma'(z) = \sigma(z)\cdot(1-\sigma(z))$
  → interpretable σ = prob of $y=1$

sign of w_1 → +ve / -ve

magnitude of w_1 →
$\|w\|$ ↑↑ logistic fⁿ becomes steeper.

- NOTE: Linear separability is considered in the input space.
- Decision boundary:
  for prediction $y=1$: $P(Y=1|X) > P(Y=0|X)$
  Threshold: $P(Y=1|X) = P(Y=0|X) = \frac{1}{2} \Rightarrow \frac{1}{1+e^{-W^TX}} = \frac{1}{2} \Rightarrow W^TX = 0$ → deci^n boundary
- MLE: likelihood funcⁿ & prob. of observing specific values given some parameters.
  comparing to LR: maximizing conditional likelihood is same as minimizing $\sum \varepsilon_i^2$ (LR assumpⁿs should hold).

$w_0^*, w_1^*, w_2^* = \underset{w_0,w_1,w_2}{\text{argmax}}\ p(y_i|x_i ; (w_0,w_1,w_2))$ ← maximizing conditional likelihood.

- Binary cross entropy:

$$w_0^*, w_1^*, w_2^* = \text{argmin} -\sum_i \left[y_i \ln(\sigma(x_i)) + (1-y_i)\ln(1-\sigma(x_i)) \right]$$

→ convex
→ no closed form solution.

$\frac{\partial J}{\partial w_0} = \sum \varepsilon_i(-1)$, $\frac{\partial J}{\partial w_1} = \sum \varepsilon_i(-x_1)$, $\frac{\partial J}{\partial w_2} = \sum \varepsilon_i(-x_2)$, ... and so on. } same as linear regression (generalized linear models).

- odds $= \frac{p}{1-p}$ for logistic reg., $\log\left(\frac{p}{1-p}\right) = W^TX$ ← linear w.r.t. parameters
- multi-class classificaⁿ: $P(y=K|X) = \text{softmax}(z_K) = \frac{e^{xw_K}}{\sum_{i=1}^{p} e^{xw_i}}$

log-likelihood: $L(X,W) = \sum_{i=1}^{n}\sum_{j=1}^{p} 1\{y_i=K\} \ln P(y_i=K|X_i)$

↳ indicator = 1 if $y_i=K$, = 0 if $y_i \neq K$

Decision Trees

- Gini Index: $Gini(D) = 1 - \sum_{i=1}^{n} p_i^2$

  → ranges from $[0, 1)$

  p_i → proportion of labels in class i.

  0 → all instances belong to single class. (least impurity)
  1 → all instances are divided equally among all classes.

- Entropy: $H(D) = -\sum_{i=1}^{K} p_i \lg p_i$

  min = 0 (all instances in same class) max = $\lg K$

  K → # of classes.

- Information Gain: $IG(D, A) = H(D) - \sum_{v \in Values(A)} \frac{|D_v|}{|D|} \cdot H(D_v)$

  A → feature on which we're splitting D

  D_v → subset for which $A = v$.

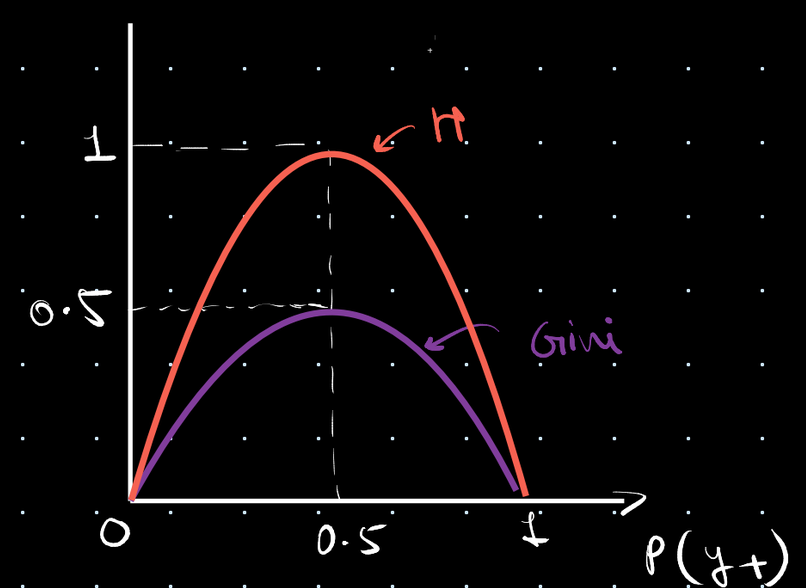
  $|D|$ → # of features in D.

  prev. entropy − weighted sum of entropies on splitting D on feature A.

  → we split on $\underset{A}{\operatorname{argmax}}\ IG(D, A)$

- Algo for top-down approach:
  → Start at root.
  → Split on best feature.
  → Repeat on all datasets.

- Stopping criteria:
  → max. depth
  → min. samples per leaf
  → min. IG
  → max. # of leaf nodes (not really good)



- Handling numerical data:
  → Consider each unique value of feature as threshold for splitting.
  → For each potential split, calculate IG. Choose the threshold that gives highest IG.

- For regression:
  → For each leaf, predicted value = mean of target values.
  → Best split → that which minimizes MSE.

- Pruning
  - pre-pruning (early stopping)
  - post-pruning
    - CCP (cost complexity pruning / weakest link pruning)
    - reduced error pruning.
    - minimum impurity decrease.
    - min. leaf size.

Ensembling

- Bagging:
  - → Do sampling with replacement (bootstrapping) k times and train classifiers $m_1, \ldots, m_k$.
  - → m^* takes majority vote and predicts class.

$$\text{out-of-bag sample} = \frac{\#\text{ of data points not in bag in bootstrap}}{\#\text{ of data points in dataset}}$$

$$\text{Bagging fraction} = \frac{\#\text{ of unique data points in bag}}{\#\text{ of data points in dataset}} \quad \underset{(N \to \infty)}{\approx} 63.2\% \text{ if } N \text{ is large}$$

→ Bagging reduces variance without affecting the bias.
so low bias, high variance base models are used. (E.g well-grown decision trees)

- Boosting:
  - → we use low variance, high bias base models + additive combining. (E.g. Decision stumps).
  - → Adjust weights of observa"s based on errors made by the prev. model.

Support Vector Machine

- We try to obtain the maximum margin hyperplane. $\max d(\pi_1, \pi_2)$ i.e. $\max \frac{2}{\|w\|}$.
- Hard margin classifier:

$$\min \frac{\|w\|}{2} \quad \text{s.t.} \quad (y_i)(w^T x_i + b) \geq 1$$

Same as perpendicular bisector of smallest distance of 2 classes.

- Dual form:

$$L = \max \left(\sum_{i=1}^{n} \alpha_i - \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j \, y_i y_j \, (x_i^T x_j) \right)$$

$\alpha > 0$ for support vectors

$$f(x_q) = \sum_{i=1}^{n} \alpha_i \, y_i \, (x_i^T x_q) + b$$

- Kernel functions:

$K(x, x') = \phi^T(x) \, \phi(x')$ ← same transformation on both the vectors.

Mercer theorem to check validity:

- $K(x, x') = K(x', x)$ → symmetry
- Kernel matrix is PSD. (Only use when kernel matrix is given, don't calculate kernel matrix, it takes time).

$$\begin{array}{c} \\ 1 \\ 2 \end{array} \begin{array}{c} \begin{array}{cc} 1 & \quad 2 \end{array} \\ \begin{bmatrix} K(x, x) & K(x, x') \\ K(x', x) & K(x', x') \end{bmatrix} \end{array}$$

NOTE: $\frac{1}{K}$ is never valid.

→ Polynomial kernel: $K(x, x') = (x^T x' + c)^d$

dim of transformed vec = # of terms = $(d+m) C_m$

$m \rightarrow$ dim of x, x'.

→ Gaussian kernel (Radial basis function): $K(x, x') = e^{\frac{-d^2}{2\sigma^2}} = e^{\frac{-\|x - x'\|^2}{2\sigma^2}}$

$\sigma \rightarrow$ bandwidth

$\sigma \uparrow\uparrow$ points far away will also be called similar.

$\sigma \downarrow\downarrow$ model becomes more complex.

- Soft margin classifier:

→ allows some misclassifications.

→ ζ_i = measure of distance for an incorrectly classified point from its correct hyperplane in incorrect direc^n.

$$\min \frac{\|w\|}{2} + C \frac{\sum \zeta_i}{n} \quad \text{s.t.} \quad (y_i)(w^T x_i + b) \geq 1 - \zeta_i$$

→ low C: large min margin, allows more misclassifica^n. overfitting ↓↓.

high C: small min margin, allows less misclassifica^n. overfitting ↑↑.

$C = 0$: all points become support vectors. margin can go to ∞.

$C = \infty$: hard margin

→ Dual form:

support vecs: $\alpha \geq 0$ for margin points.

$\alpha = C$ for misclassified points. $\alpha \in [0, C]$

- Hinge loss formulation: $\min \left\{ \sum \left[\max(0, 1 - y_i(w^T x_i + b)) \right] + \lambda \|w\|^2 \right\}$

↑ Hinge loss

Clustering
- Hard (point completely belongs to a cluster)
- Soft (prob. of point belonging to cluster is evaluated)

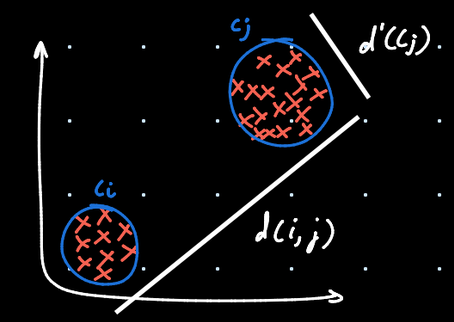
K-means clustering:
- Steps:
  - → Initialize k centroids. (random or using smth. like K-means++).
  - → Assign each data point to nearest centroid.
  - → Calculate new centroid of each cluster.

  } Repeat until convergence

$$\text{cluster}(x_i) = \arg\min_K d(x_i, c_K)$$

$$\text{centroid update } c_K = \frac{1}{N_K} \sum_{x_i \in C_K} x_i$$

Dunn Index

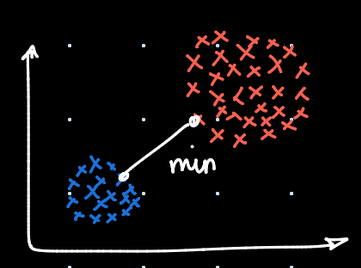


$$D = \frac{\min_{i,j} d(i,j)}{\max_K d'(K)}$$ } minimal inter-class dist. / maximal intra-class distance.

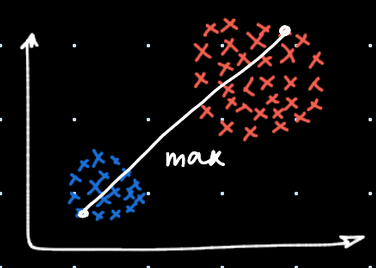
- K-medoids clustering:
  medoid is a point from the dataset from which sum of distances to other points is minimal.

- Hierarchical clustering:

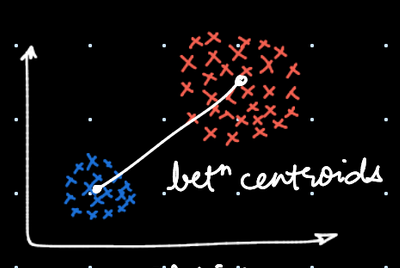
  Linkage criteria:



Single linkage
$= \min\{d(A_i, B_j)\}$



Complete linkage:
$= \max\{d(A_i, B_j)\}$



Centroid linkage:
$= d(C_A, C_B)$

$$\text{Avg. linkage} = \frac{1}{|A|\,|B|} \sum_{a \in A} \sum_{b \in B} d(a,b)$$

Ward's linkage:

$$W(A) = \sum_{x \in A} \|x - \mu_A\|^2$$

ward's distance is the increase in variance when merging clusters A and B.

$$\Delta W = W(C) - [W(A) + W(B)] = \frac{|A| \cdot |B|}{|A| + |B|} \|\mu_A - \mu_B\|^2$$

↳ dist. betn centroids

- Agglomerative hierarchical clustering: bottom-up
  - → Each point is a separate cluster (n clusters).
  - → Calculate pairwise distance betn all clusters (matrix).
  - → Merge closest clusters.
  - → Update distance matrix.

  } repeat till single cluster.

Elbow method: Plot within cluster sum of squares for diff. K values, choose K at elbow point.

- Divisive Hierarchical clustering (top-down):
  - → Consider all points in a single cluster.
  - → Split into clusters using smth. like K-means.
  - → Among these, choose with largest sum of squares to split further.

  } Repeat.

- Density-based clustering:
  - → group closely packed points. Mark outliers as low-density regions.
  - → Can figure out arbitrary-shaped clusters.

Model Evaluation

- Bias = $E[\hat{y}] - y$

- High bias:
  - → model underfits.
  - → large training error.
  - To reduce:
  - → increase model complexity.
  - → add more features.
  - → reduce regularization.
  - → Increase training data.
  - → feature engineering.

- High variance:
  - → model overfits. (overly complex model, fits noise).
  - → Large test error.
  - → model sensitive to small changes in data.
  - To overcome:
  - → reduce complexity
  - → use regularizaⁿ.
  - → Increase data.
  - → Ensemble methods (bagging).
  - → pruning (in decision trees) or other stopping criteria.

- Error = $bias^2$ + variance + irreducible error (σ^2)

- cross validation:

  Holdout: make 3 sets: train, validation (use for hyperparameter tuning), test.

  leave-one-out-cross validation (LOOCV):
  - → Each point acts as validation point once, remaining form train set. This is repeated for all points.

  k-fold cross validation:
  - → Dataset is divided into k folds. Each fold acts once as the validaⁿ set while the remaining (K-1) act as train sets.
  - → k = 5/10. K↑↑ variation ↑↑ bias ↓↓
  - → Performance metric is averaged across k iteraⁿs.

LOOCV	K-folds	Holdout
$N(N-1)$	$k(N-\frac{N}{k})$	$1 * (N * 0.8)$
↑ models ↑ Dtrain	↑ model ↑ Dtrain	↑ model ↑ Dtrain

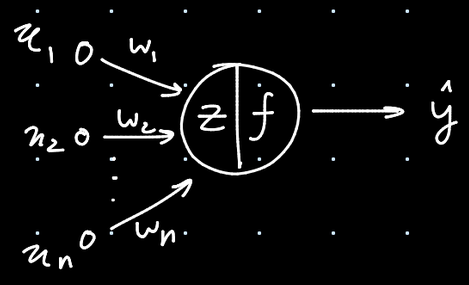
Stratified CV:
- → Stratified sampling: class proportion in sample is same as that in population.
- → The folds need not be mutually exclusive.

Misc stuff

- Generative model → models $P(X|Y) \cdot P(Y)$, learns joint distribution.
- Discriminative model → models $P(Y|X)$ directly.

Neural Networks

- Artificial Neuron:

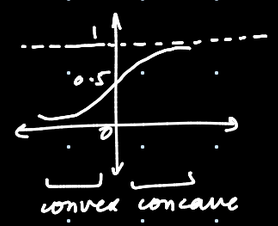


- Activation functions:

Sigmoid:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$

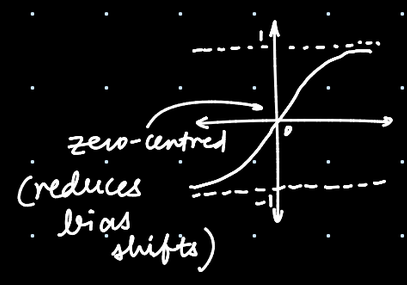


tanh:

$$\tanh(z) = \frac{e^z + e^{-z}}{e^z - e^{-z}}$$

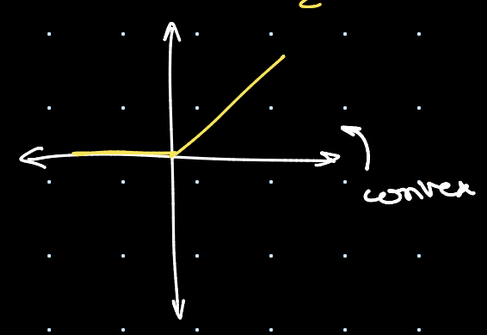
$$f'(z) = 1 - (f(z))^2$$

$$\tanh(z) = 2 \cdot \sigma(2z) - 1$$



ReLU:

$$z^+ = \max(0, z) = \frac{z + |z|}{2}$$



Softmax

$$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- Loss functions:

$$MSE = \frac{1}{n}\sum(\hat{y}_i - y)^2 \qquad MAE = \frac{1}{n}\sum(\hat{y}_i - y_i)^2$$

$$BCE = \frac{-1}{N}\sum_{i=1}^{N}\left(y_i \lg(\hat{y}_i) + (1-y_i)\lg(1-\hat{y}_i)\right)$$

$$CCE = \frac{-1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C} y_{ij} \lg(\hat{y}_{ij})$$

$$\text{Hinge loss} = \frac{1}{N}\sum_{i=1}^{N}\max(0, 1 - y_i \hat{y}_i)$$

- Feed-Forward Neural Networks:

$$\#\text{ of weights for an entire layer} = (n_{prev} + 1) \cdot n_{curr}$$

Vanishing gradient:

- due to tanh, sigmoid like activation functions. $\xrightarrow{\text{soln}}$ ReLU, batch normalization, better wt. initialization; ResNets.

- GD: $w_{new} = w_{old} - \eta \frac{\partial L}{\partial w_{old}}$ (or batch GD)

SGD: Uses a single point at a time.

Mini-batch: uses a set of random points.

GD → surely converges to global minimum given a convex function and optimal learning rate.
Slower, costlier, esp. for large datasets.

SGD → Really noisy, requires much more iterations, but the iterations are much faster.
may never converge.

MB-SGD → May never converge (theoretically).

Learning rate too high → any of these may never converge.